TK101 AI Platform 기술 검토 보고서

프로젝트	TK101 사내 AI 자동화 플랫폼
작성일	2026-04-16
담당자	TK101 (junki7853)
현재 단계	Plan & Design 완료 → Sprint 0 착수 직전
목적	본인 기술 검토용 — 아키텍처 결정 및 향후 작업 흐름 확인

목차

1. 프로젝트 개요 및 진행 현황
2. 기술 스택 확정
3. 아키텍처 결정 — Clean Architecture 채택 근거
4. 도구 전략 — bkit + ECC 병행
5. 문서 구조 — Router Pattern 적용
6. Sprint 로드맵
7. 리스크 & 완화 전략
8. 다음 단계

1. 프로젝트 개요 및 진행 현황

1.1 프로젝트 정의

40명 규모 TK101 Global Korea Inc. 사내 AI 자동화 플랫폼. 마케팅1/마케팅2/디자인/신사업/회계 5개 팀의 업무를 AI로 자동화. 1인 담당자(TK101)가 Claude Code 기반으로 구축·유지보수.

1.2 지금까지 진행한 작업

단계	작업 내용	산출물	상태
기존 문서 검토	프로젝트 설계, 기술스택, ECC+bkit 전략, 인프라 메모, 테스트 우선순위 5개 문서 종합 분석	요구사항 정리	완료
PDCA Plan	1단계 뼈대 구축 요구사항 · 범위 · 성공기준 문서화. 회계는 2단계로 분리.	`docs/01-plan/features/tk101-platform-foundation.plan.md`	완료
PDCA Design	3가지 아키텍처 옵션 비교 후 Clean Architecture 선택. API 명세 · DB 스키마 · 테스트 계획 포함	`docs/02-design/features/tk101-platform-foundation.design.md`	완료
프로젝트 헌법	CLAUDE.md 라우팅 테이블 중심으로 경량화. 작업 유형별 온디맨드 파일 로딩 설계	`CLAUDE.md` 90줄 + 규칙 파일 4개	완료
Phase 1 Schema 공식화	DB 스키마를 별도 문서로 정식화 (Alembic 마이그레이션 참조용)	`docs/01-plan/schema.md`	선택
Phase 2 Convention 공식화	코딩 컨벤션 문서화 (ESLint/Prettier 포함)	`docs/01-plan/conventions.md`	선택
Sprint 0 구현 착수	프로젝트 디렉터리 구조, FastAPI/Next.js 초기화, Docker Compose	backend/, frontend/, docker-compose.yml	대기

1.3 인프라 현황

- Tencent Cloud CVM 4vCPU/8GB RAM — 생성 완료
- GitHub 레포지토리 — 생성 완료, 로컬 git 연결 완료
- GitHub Actions → CVM 자동 배포 — 미설정 (Sprint 1에서 구축)
- API 키 보유: Claude / OpenAI / Gemini

2. 기술 스택 확정

영역	선택	선택 근거
백엔드	**FastAPI** (Python 3.12+, async)	AI 라이브러리 생태계 · Claude Code의 Python 생성 품질 · Pydantic 자체 문서화
프론트엔드	**Next.js 15** (App Router, TypeScript)	SSR · 미들웨어 · API Routes 내장. 채팅 UI와 백오피스를 단일 프로젝트로 관리
DB	**PostgreSQL 16** + pgvector	JSON 타입(AI 응답 저장) · 향후 RAG 대응 · 회계 데이터 트랜잭션 안정성
ORM	**SQLAlchemy 2.0 async** + Alembic	타입 힌트 · async 지원 · 마이그레이션 관리
캐시/큐	**Redis**	세션 · Rate limit · 향후 작업 큐
인증	**JWT** (HS256) + bcrypt	프론트-백 분리 구조 · 모바일 확장 대비 · access 30분/refresh 14일
스타일	**Tailwind + shadcn/ui**	빠른 UI 구축 · 컴포넌트 재사용
상태 관리	TanStack Query (서버) + Zustand (클라이언트)	서버 상태와 클라이언트 상태 명확히 분리
테스트	pytest + httpx (BE), Playwright (FE E2E)	커버리지 80% 이상 목표
컨테이너	**Docker Compose**	40명 규모에 Kubernetes는 과도. 단일 서버 Compose로 충분.
배포	GitHub Actions → Tencent CVM	자동 CD. 서버 직접 접속 없이 git push로 배포 완료

3. 아키텍처 결정 — Clean Architecture 채택 근거

3.1 3가지 옵션 비교

기준	A: Minimal	B: Clean Architecture ★	C: Pragmatic
구조	평면 폴더	4-layer 엄격 분리	Feature-based 모듈
새 파일 수	~25	~60	~40
복잡도	낮음	높음	중간
유지보수성	낮음 (3년 시점 부담)	**높음 (3년 이후에도 견고)**	높음
초기 세팅 시간	1일	3일	2일
회계 모듈 정확도	낮음 (DB 의존 테스트)	**높음 (Domain 순수 테스트)**	중간
멀티 AI 전환 용이성	낮음	**높음 (Adapter 패턴)**	중간

3.2 B 선택 결정 이유

> **핵심 판단:** 4/21 미팅 시연 제약을 배제할 수 있다면(→ 5월 중순까지 회계 완성 가능), 장기 관점에서 B가 명확히 우위. Claude Code로 유지보수하는 전제에서도 초기 세팅 3일 투자가 3년 운영 혜택으로 상환됨.

1. **회계 로직 정확도 담보** — 은행 파싱/세금계산서 매칭은 돈을 다루는 로직. Domain 레이어 순수 함수로 DB 없이 단위 테스트 가능 → 이사님께 "매칭 정확도 검증됨" 당당히 보고
2. **멀티 AI 전환 대비** — 3단계 OpenAI/Gemini 추가 계획. Infrastructure 레이어 Adapter 패턴으로 Claude → 타 AI 교체 시 다른 코드 수정 최소화
3. **5개 팀 모듈 확장성** — 마케팅1/2, 디자인, 신사업, 회계 각 모듈이 bounded context. 패턴이 확립되면 Claude Code가 새 모듈을 빠르게 복제 가능
4. **END USER 비용 무관** — 레이어 증가가 API 호출 경로를 늘리지 않음. 직원 토큰 비용은 아키텍처와 독립
5. **테스트 커버리지 80%+ 달성 용이** — Domain 레이어는 외부 의존성 0 → TDD가 매우 쾌적

3.3 4 레이어 구조

```
┌──────────────────────────────────────────────────────────┐
│  Presentation (app/api/)                                  │
│    FastAPI routers, Pydantic schemas, middleware          │
├──────────────────────────────────────────────────────────┤
│  Application (app/use_cases/)                             │
│    Use Case 클래스 — 비즈니스 흐름 오케스트레이션          |
├──────────────────────────────────────────────────────────┤
│  Domain (app/domain/) ★ 외부 의존성 0                      │
│    Entity, Value Object, Repository 인터페이스, Domain Service |
├──────────────────────────────────────────────────────────┤
│  Infrastructure (app/infrastructure/)                     │
│    SQLAlchemy 구현, Claude Adapter, JWT, Redis, bcrypt    |
└──────────────────────────────────────────────────────────┘


의존 방향:
  Presentation → Application → Domain ← Infrastructure
                                        (Dependency Inversion)
```

3.4 Import 규칙 (절대 규칙)

From	Import 가능	Import 금지
Presentation	Application, Domain	Infrastructure 직접 (DI로만)
Application	Domain	Presentation, Infrastructure 구현체
Domain	**외부 의존성 0**	**FastAPI, SQLAlchemy, pydantic 전부 금지**
Infrastructure	Domain	Application, Presentation

4. 도구 전략 — bkit + ECC 병행

4.1 역할 분담

- **bkit** = 프로세스 매니저 (PDCA 흐름, 기획, QA 단계, 배포 관리)
- **ECC** = 기술 전문가 (언어별 코드 리뷰, 보안, TDD, 빌드 해결)
- 중첩 금지: 같은 작업을 두 도구로 동시에 하지 않음

4.2 표준 13단계 플로우 (기능 1개 개발 시)

#	단계	명령어	주도
1	기획	`/bkit:pdca plan <feature>`	bkit
2	설계 (아키텍처 옵션 선택)	`/bkit:pdca design <feature>`	bkit
3	UI 목업 (선택)	`/bkit:phase-3-mockup`	bkit
4	구현 계획 세부화	`/plan <feature>`	ECC
5	테스트 먼저 작성 (L0~L3)	`/tdd <feature>`	ECC
6	코드 작성 (자동 리뷰)	저장 시 `python-reviewer` / `typescript-reviewer` 자동	ECC
7	구현 진행 추적	`/bkit:pdca do <feature> --scope X`	bkit
8	QA 실행	`/bkit:qa-phase <feature>`	bkit + ECC e2e-runner
9	갭 분석 (90%+ 목표)	`/bkit:pdca analyze <feature>`	bkit
10	자동 개선 (필요시)	`/bkit:pdca iterate <feature>`	bkit
11	완료 보고서	`/bkit:pdca report <feature>`	bkit
12	배포	`git push` → GitHub Actions 자동	-
13	아카이브	`/bkit:pdca archive <feature> --summary`	bkit

4.3 상세 도구 선택 매트릭스 (요약)

작업 상황	1차 도구
새 기능 기획 시작	`/bkit:pdca plan`
FastAPI 코드 작성	ECC `python-reviewer` (자동)
Next.js 코드 작성	ECC `typescript-reviewer` (자동)
인증/보안 코드 (반드시)	ECC `security-reviewer`
빌드 에러	ECC `build-error-resolver`
QA 테스트	`/bkit:qa-phase` + ECC `e2e-runner`
배포	`/bkit:deploy` + ECC `github-ops` (설정)

상세 내용: `.claude/rules/project/tool-split.md`

5. 문서 구조 — Router Pattern 적용

5.1 설계 원칙

CLAUDE.md를 매 세션 자동 로드되는 "헌법"으로 삼되, 내용 희석 방지를 위해 최소한의 라우팅 테이블과 절대 규칙만 포함. 상세 내용은 요청 유형에 따라 Just-in-Time 로드.

5.2 파일 구조

```
TK101 AI/
├── CLAUDE.md (약 90줄) ← 매 세션 자동 로드
│   ├── §1 프로젝트 한 줄 정의
│   ├── §2 절대 규칙 (Clean Architecture Import 규칙 등)
│   ├── §3 작업 라우팅 테이블 ★ 핵심
│   ├── §4 세션 시작 3단계 체크리스트
│   └── §5 참조 문서 링크
│
├── .claude/rules/project/ ← 필요 시만 Read
│   ├── tool-split.md        (83줄)
│   ├── workflow.md          (198줄)
│   ├── commands-cheatsheet.md (185줄)
│   └── current-status.md    (45줄, 자주 업데이트)
│
├── docs/
│   ├── 01-plan/features/tk101-platform-foundation.plan.md
│   └── 02-design/features/tk101-platform-foundation.design.md
│
└── 초기설정 구성방안/ (원본 기획 문서 5개)
```

5.3 라우팅 매커니즘 예시

> **사용자 요청:** "로그인 구현해줘"
> **라우팅:** CLAUDE.md §3 참조 → "구현 시작" 매핑 → Design 문서 §9 + workflow.md §B 로드
> **효과:** 불필요한 명령어 치트시트나 UI 목업 가이드는 로드되지 않음 → 컨텍스트 집중

5.4 희석 방지 원칙

- 구체적 규칙 > 추상적 규칙 (예: "파일 ≤ 400줄" 은 도구로 검증 가능)
- 중복 제거 (같은 정보를 두 곳에 두지 않음)
- 동적 vs 정적 분리 (현재 진행 상태는 별도 파일)
- 명령형 표현 (권장형 "고려하세요" 금지)

6. Sprint 로드맵

Sprint	기간	목표	핵심 산출물
Sprint 0	4/16~4/18 (3일)	아키텍처 골조 세팅	Clean Architecture 4 레이어 스켈레톤, Docker Compose, Next.js 초기화
Sprint 1	4/19~4/25 (1주)	인증 + 대시보드 쉘 + CI/CD	로그인 API, 로그인 페이지, 대시보드 쉘, GitHub Actions
📌 4/21 월요일	-	**이사님 미팅 시연**	아키텍처 발표 + 로그인 데모 + CI/CD 시연
Sprint 2	4/26~5/9 (2주)	백오피스 + AI 채팅 + 보안	사용자/부서 CRUD, AI 채팅 UI, Rate limiting, HTTPS
뼈대 완성	5/9	1단계 완료	테스트 커버리지 80%+, 보안 리뷰 통과
Sprint 3 (별도 PDCA)	5/10~5/중순	회계 자동화 모듈	은행 파싱, 세금계산서 매칭, 엑셀 리포트

6.1 4/21 미팅 시연 시나리오

1. 아키텍처 다이어그램 발표 — Clean Architecture 4 레이어 + 5개 팀 모듈 확장 계획
2. Docker Compose 실행 시연 — `docker compose up` 으로 전체 스택 기동
3. GitHub Actions 시연 — 코드 push → CVM 자동 배포 로그 확인
4. 로그인 페이지 데모 — 로그인 성공 후 대시보드(빈 쉘) 진입
5. Key Message: "견고한 설계에 투자하여 5개 팀 모듈을 안정적으로 올릴 기반 구축. AI 채팅은 다음주, 회계 자동화는 5월 중순 완성 예정."

7. 리스크 & 완화 전략

리스크	영향	완화책
1인 개발자 병목	높음	bkit+ECC 병행으로 PM + 시니어 역할 동시 확보. Claude Code 멀티 에이전트로 병렬 작업
Clean Architecture 초기 세팅 부담	높음	Sprint 0 3일 전용 배정. 이후 패턴 확립되면 속도 향상
4/21 미팅 기대치 관리	중간	"설계+로그인 데모" 수준으로 기대치 조정. 회계는 5월 중순 완성 명시
CVM 단일 서버 장애	중간	Docker Compose restart policy + 일일 DB 백업. 이중화는 2단계
Claude API 비용 초과	중간	모델 라우팅 (Haiku 80%), 부서별 사용량 추적, 월별 한도
비개발자 직원의 낮은 기술 이해도	중간	직관적 UI. 로그인 외 복잡한 설정 없이 사용
GitHub Secrets/API 키 유출	높음	.gitignore, GitHub Secrets, security-reviewer 자동 스캔

8. 다음 단계

8.1 즉시 실행 가능한 3가지 경로

옵션	명령어	기대 결과
A. Phase 1 Schema 공식화	`/bkit:phase-1-schema`	DB 스키마 별도 문서 생성. Alembic 마이그레이션 직접 연결
B. Phase 2 Convention 공식화	`/bkit:phase-2-convention`	ESLint/Prettier 포함 코딩 컨벤션 정식 문서
C. Sprint 0 즉시 착수 (권장)	`/bkit:pdca do tk101-platform-foundation --scope skeleton`	프로젝트 디렉터리, FastAPI/Next.js 초기화, Docker Compose 시작

> **권장:** C로 즉시 착수. Sprint 0 skeleton 단계에서 Alembic 마이그레이션을 작성하면서 Phase 1 스키마 문서가 자연스럽게 도출됨. 시간 압박 상황에서 문서 선행보다 구현 중 문서화가 실용적.

8.2 세션당 작업 단위 권장

- 1 세션 = 1 scope (`skeleton`, `domain-db`, `auth` 등)
- 세션당 실제 코드 변경 300~500줄

- 세션 시작 시 CLAUDE.md 자동 로드 + 현재 feature Design 수동 로드